

3-dars: Memory Management

There is a `struct` called a `PyObject`, the **grand-daddy of all objects in Python**, contains only two things:

- `ob_refcnt`: reference count - murojaatlar soni
- `ob_type`: pointer to another type

Garbage Collector = agar ref_cnt 0 ga teng bo'lsa o'sha obyektни o'chiradi!

cpython/obmalloc.c at 7d6ddb96b34b94c1cbdf95baa94492c48426404e · python/cpython

This file contains bidirectional Unicode text that may be interpreted or compiled differently than what appears below. To review, open the file in an editor that reveals hidden Unicode characters. Learn more about bidirectional Unicode characters

<https://github.com/python/cpython/blob/7d6ddb96b34b94c1cbdf95baa94492c48426404e/Objects/obmalloc.c>

python/**cpython**

The Python programming language

2k

Contributors

1

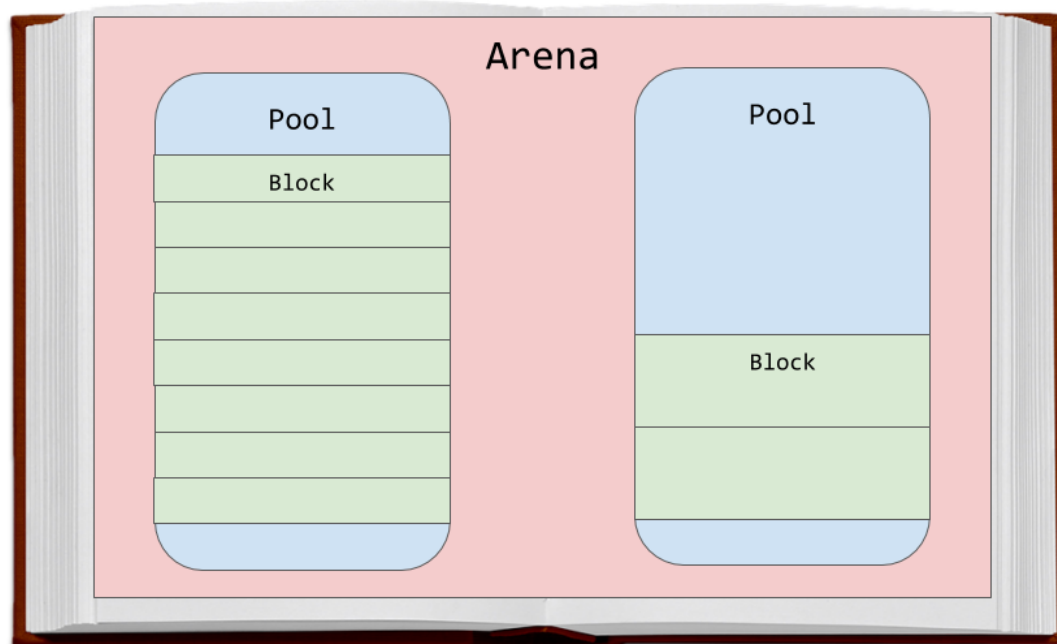
Used by

51k

Stars

26k

Forks

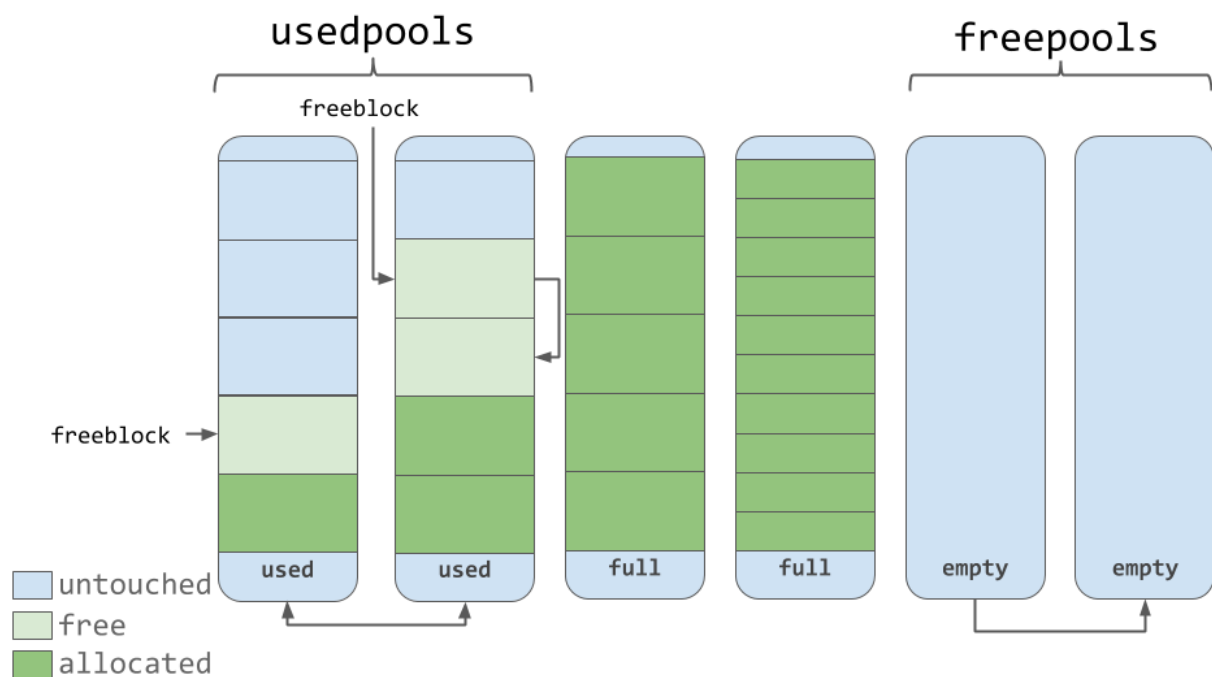


Arena - 256 kilobytes

pool - 4 kilobytes

Request in bytes	Size of allocated block	Size class idx
1-8	8	0
9-16	16	1
17-24	24	2
25-32	32	3
33-40	40	4
41-48	48	5
49-56	56	6
57-64	64	7
65-72	72	8

Pools themselves must be in one of 3 states: `used`, `full`, or `empty`



- **untouched** : a portion of memory that has not been allocated
- **free** : a portion of memory that was allocated but later made “free” by CPython and that no longer contains relevant data
- **allocated** : a portion of memory that actually contains relevant data

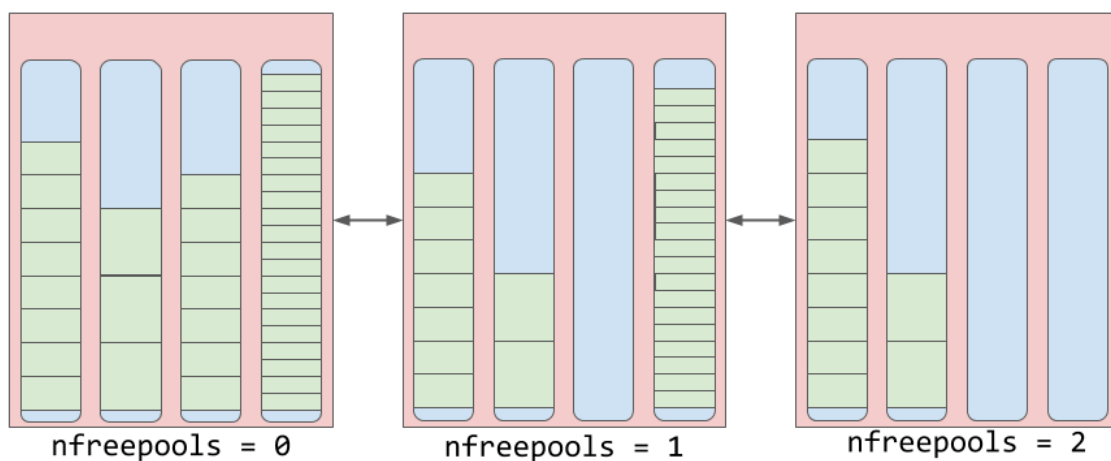
freeblocks -linked list

pools -double-linked list

arenas - double-linked list

ARENAS

`usable_arenas`



GIL - Global Interpreter Lock

by Bobosher Musurmonov

Concurrency bu bir nechta **thread**(oqim)larning bitta processorda ishlashi. Umuman olganda, threadlar processorda juda kichik instructionlarga ajratilib(Assemblyga o'xshash), navbatma-navbat bajariladi. Bu instruksiyalar juda kichkina bo'lgani va processor bir soniyada bunaqa instruksiyalardan milliardlab bajargani uchun bizga xuddi bir vaqtda ishlayotgandek taassurot qoldiradi.

Concurrency paytida ba'zi o'zgaruvchilarni ikkita thread bir vaqtda o'zgartirishi, buning orqasidan noto'g'ri natijalar kelib chiqishi mumkin(masalan, `a=0` o'zgaruvchiga 2 marta 1 qo'shib, natija 1 chiqishi mumkin). Mana bu **race condition** deyiladi.

Race conditionning oldini olishning bir usuli sifatida **lock** o'ylab topilgan. Lockning ishlashi quyidagicha: biror thread biror o'zgaruvchining qiymatini o'zgartirmoqchi bo'lgan paytda uni "qulflab" qo'yadi. Shunda o'zgaruvchini bir vaqtda faqat bitta thread o'zgartira oladi. Bu qiymatni olmoqchi bo'lgan boshqa thread qulf ochilishini kutib, ochilgandan keyin yangi qiymatni olib ishlataveradi.

Lock juda yaxshi yechim, lekin bitta jiddiy muammosi bor. Deylik, bitta thread a o'zgaruvchini "lock" qilgan va b ning qiymatini olmoqchi. Ikkinchi thread b ni "lock" qilgan va a ning qiymatini olmoqchi. Shunda ikkita thread yopiq sikl hosil qilib bir-birini qulflab qo'yadi. Bu esa dastur o'sha joyda "tiqilib qolishi"ga olib keladi. Mana bu holat "**dead lock**" deyiladi.

Pythonda xotira boshqaruvi avtomatik bajariladi. Ya'ni qaysidir qiymat ishlatilmasa, python uni avtomatik aniqlab, xotiradan bo'shatadi. Buni amalga oshirish uchun python **reference count** degan narsadan foydalanadi. Sodda qilib aytganda, o'sha qiymatdan necha kishi foydalanayotganini sanab turadi. O'sha raqam 0 ga tushganda(demak, hech kim ishlatmayapti) garbage collector o'sha qiymatni xotiradan o'chiradi.

Race condition hamma joyda sodir bo'lishi mumkin. Shu jumladan **reference count**da ham. Lekin bu yerda sodir bo'lsa kerakli ma'lumotlar o'chib ketishi yoki keraksiz ma'lumot xotirada qolib ketishi kabi muammolar kelib chiqadi. Buning oldini olish uchun Python reference count uchun ham lock ishlatadi. Lekin yuqorida aytilganidek, dead lock muammosini ham hisobga olish kerak. Shuning uchun Python hamma uchun bitta **GIL(Global Interpreter Lock)** ishlatadi. Bu ham oddiy lock kabi ishlaydi, lekin u interpreterda saqlanadi. Qaysidir thread biror o'zgaruvchini o'zgartirmoqchi bo'lsa interpreter GILni o'sha threadga vaqtinchalik berib turadi. Thread esa undan o'zgaruvchini emas, uning qiymatidagi reference countni qulflashda foydalanadi. Ishini tugatib bo'lgach, u GILni yana interpreterga qaytaradi va interpreter GILni boshqa threadga beradi... Butun interpreterda bitta GIL bo'lgani uchun dead lock sodir bo'lmaydi. Sababi, qulf bitta, hech kim bir-birini qulflab qo'ymaydi.

Demak, GIL bizning dasturimizdagi o'zgaruvchilar bilan bo'ladigan race conditionning oldini olish uchun emas, Pythonning ichki, xotira boshqaruv tizimi to'g'ri ishlashi uchun xizmat qiladi.